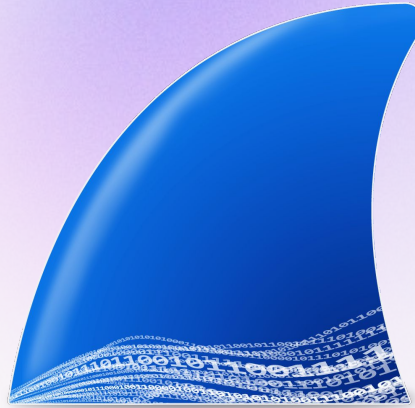


# Soutenance 10

## WIRESHARK

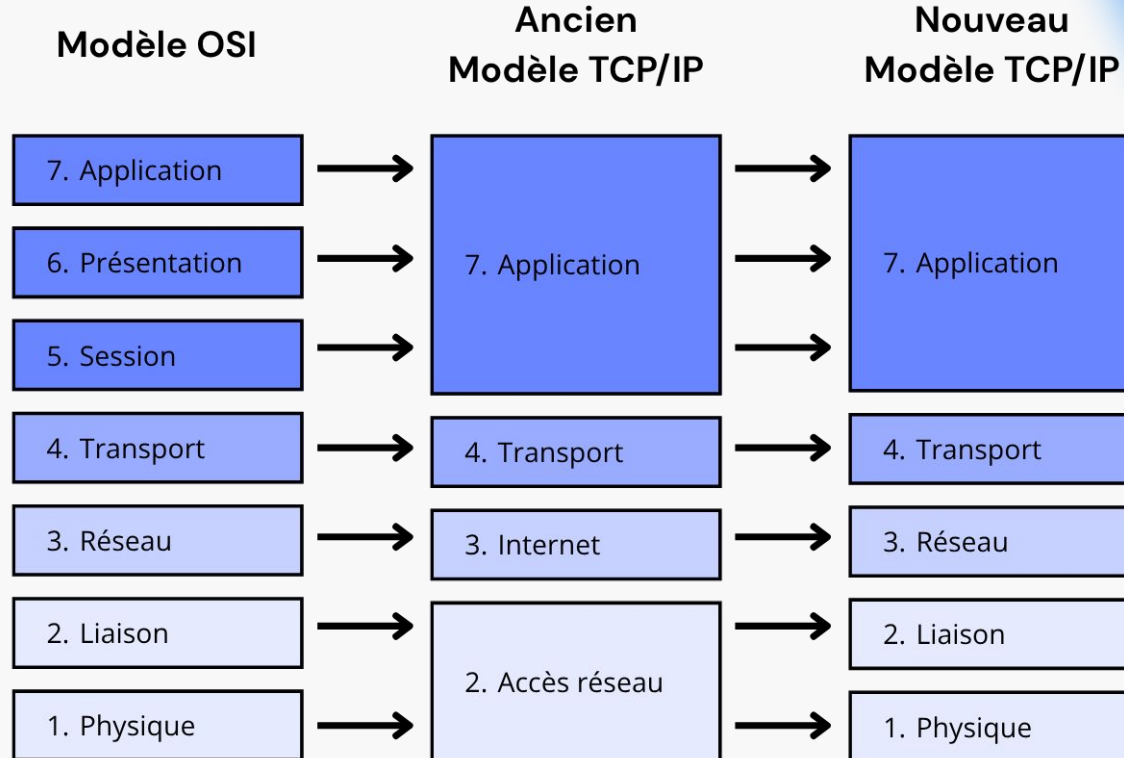


# Qu'est ce que Wireshark ?

Wireshark est un sniffer de paquets open source et gratuit, qui capture et examine en temps réel le trafic réseau. Développé en 1998 par Gerald Combs sous le nom d'Ethereal, il est aujourd'hui l'un des analyseurs de paquets le plus utilisé au monde.

Détection des failles de sécurité, analyse des performances réseau, développement de protocoles de communication... grâce à sa polyvalence, Wireshark est devenu l'allié incontournable de nombreux experts en cybersécurité.

# Tout d'abord il est important de comprendre le modèle OSI



01 | Qu'est ce que le protocole **ARP** ?

02 | Qu'est ce que le protocole **TCP** ?

03 | Qu'est ce que le protocole **UDP** ?

## 01 ARP, le protocole de résolution d'adresses

ARP (**Address Resolution Protocol**) traduit les adresses IP en adresses MAC et vice versa afin que les appareils du réseau local puissent communiquer entre eux. L'ARP est nécessaire parce que les adresses IP et MAC sont de longueurs différentes et fonctionnent sur des couches différentes du modèle OSI. Vous trouverez ci-dessous une ventilation des différentes longueurs d'adresses :

- Les adresses **IPv4** sont de 32 bits.
- Les adresses **IPv6** sont de 128 bits.
- Les adresses **MAC** – le numéro physique d'un appareil – sont composées de 12 chiffres hexadécimaux répartis en six paires.

Ces adresses doivent être traduites pour permettre une communication et un transfert de données corrects entre les appareils connectés. L'ARP n'est pas nécessaire à chaque fois que des périphériques tentent de communiquer, car le système hôte du réseau local cartographie et stocke les adresses traduites dans son Cache ARP.

Par conséquent, le processus de traduction ARP est principalement utilisé lorsque de nouveaux appareils rejoignent le réseau.

## 02 TCP, le protocole de contrôle de transmission

TCP (**Transmission Control Protocol**) est une couche de transport et constitue l'autre moitié du TCP/IP. Il organise les paquets dans l'ordre pour que le protocole IP puisse les acheminer. Plus précisément, le TCP numérote les paquets individuels parce que le protocole IP peut envoyer des paquets vers leurs destinations par des routes différentes et les mettre dans le désordre.

TCP détecte également les erreurs dans le processus d'envoi – notamment si des paquets sont manquants d'après le système de numérotation de TCP – et demande à IP de retransmettre ces paquets avant que l'IP ne livre les données à leur destination. Grâce à ce processus, la suite TCP/IP contrôle la communication sur Internet.

## 03 UDP, le protocole de datagramme utilisateur

UDP (**User Datagram Protocol**) est une alternative à TCP et fonctionne également avec l'IP pour transmettre des données sensibles au temps. L'UDP permet des transmissions de données à faible latence entre les applications Internet, ce qui le rend idéal pour la VoIP ou d'autres scénarios d'usage à base d'audio et de vidéo.

Contrairement à TCP, UDP n'attend pas l'arrivée de tous les paquets et n'organise pas les paquets. Au contraire, il transmet tous les paquets, même si certains ne sont pas encore arrivés.

UDP transmet uniquement des paquets, tandis que TCP transmet, organise et s'assure que les paquets arrivent. Si l'UDP fonctionne plus rapidement que le TCP, il est également moins fiable.

# 01 Questions/Réponses

- 01 Quelle est la différence entre une trame et un paquet ? Qu'est-ce que le format pcap/pcapng ?
- 02 Quelles sont les adresses MAC sources, les IP sources et les adresses MAC destinations, les IP destinations des données capturées ?
- 03 Référez d'autres trames ou paquets circulants sur le réseau. Identifiez leurs protocoles et leur fonction.
- 04 Décrivez le mécanisme de connexion avec un diagramme.

# 01 Quelle est la différence entre une trame et un paquet ? Qu'est-ce que le format pcap/pcapng ?

Une trame est utilisée pour envoyer des données à l'intérieur d'un seul réseau.

Un paquet est utilisé pour envoyer des données d'un réseau à un autre, puis à un appareil spécifique sur ce réseau.

Un fichier PCAP (Packet Capture) est un format de fichier utilisé pour enregistrer les données du trafic réseau. Il permet de stocker des paquets de données échangés sur un réseau, capturés par des outils comme Wireshark ou TCPdump.

PCAPNG permet quant à lui d'avoir une meilleure extensibilité ainsi qu'une meilleure portabilité que PCAP, il possède une structure de fichiers plus flexible et conclut également des informations supplémentaires sur le paquet et l'interface utilisé pour sa capture, telles que les compteurs de paquets perdus, les enregistrements DNS, etc...<

02

03

Référez d'autres trames ou paquets circulants sur le réseau. Identifiez leurs protocoles et leur fonction.

| Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide |              |                        |                |          |                        |
|----------------------------------------------------------------------------------------|--------------|------------------------|----------------|----------|------------------------|
| Appliquer un filtre d'affichage ... <Ctrl-I>                                           |              |                        |                |          |                        |
| No.                                                                                    | Time         | Source                 | Destination    | Protocol | Length Info            |
| 396                                                                                    | 3.895798780  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.1.22  |
| 397                                                                                    | 3.895798830  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.22  |
| 398                                                                                    | 3.895798879  | 10.10.19.172           | 224.0.0.251    | MDNS     | 107 Standard query 0x0 |
| 399                                                                                    | 3.897651230  | 142.251.209.99         | 10.10.2.35     | QUIC     | 162 Protected Payload  |
| 400                                                                                    | 3.897742373  | 10.10.2.35             | 142.251.209.99 | QUIC     | 75 Protected Payload   |
| 401                                                                                    | 3.900373319  | 142.251.209.99         | 10.10.2.35     | QUIC     | 559 Protected Payload  |
| 402                                                                                    | 3.900481044  | 10.10.2.35             | 142.251.209.99 | QUIC     | 79 Protected Payload   |
| 403                                                                                    | 3.938145225  | 142.251.209.99         | 10.10.2.35     | QUIC     | 67 Protected Payload   |
| 404                                                                                    | 3.993585179  | HewlettPacka_88:5c:... | Broadcast      | ARP      | 60 Who has 10.10.200.  |
| 405                                                                                    | 3.996439021  | 10.10.17.235           | 224.0.0.251    | MDNS     | 140 Standard query res |
| 406                                                                                    | 3.996439392  | CloudNetwork_76:2f:... | Broadcast      | ARP      | 60 Who has 10.10.18.5  |
| 407                                                                                    | 3.996439532  | WNC_c7:50:f3           | Broadcast      | ARP      | 60 Who has 10.10.18.9  |
| 408                                                                                    | 3.996439672  | 10.10.18.18            | 224.0.0.251    | MDNS     | 107 Standard query 0x0 |
| 409                                                                                    | 3.996439811  | Intel_eb:d0:c0         | Broadcast      | ARP      | 60 Who has 10.10.15.9  |
| 410                                                                                    | 3.996439947  | Intel_a1:5e:1e         | Broadcast      | ARP      | 60 Who has 10.10.19.2  |
| 411                                                                                    | 3.999405482  | 10.10.18.100           | 224.0.0.251    | MDNS     | 81 Standard query 0x0  |
| 412                                                                                    | 3.999405753  | ChinaDragonT_b9:80:... | Broadcast      | ARP      | 60 Who has 10.10.19.1  |
| 413                                                                                    | 3.999405985  | 10.10.141.132          | 224.0.0.251    | MDNS     | 119 Standard query res |
| 414                                                                                    | 3.999406042  | 10.10.16.142           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 415                                                                                    | 4.001668820  | 10.10.13.170           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 416                                                                                    | 4.096074942  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.20  |
| 417                                                                                    | 4.098508346  | SamsungElect_a4:f5:... | Broadcast      | ARP      | 60 Who has 10.10.17.1  |
| 418                                                                                    | 4.098508866  | 10.10.18.100           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 419                                                                                    | 4.098508995  | HuiZhouGaosh_94:db:... | Broadcast      | ARP      | 60 Who has 10.10.14.4  |
| 420                                                                                    | 4.098509120  | HuiZhouGaosh_94:db:... | Broadcast      | ARP      | 60 Who has 10.10.17.1  |
| 421                                                                                    | 4.098509249  | ActionsMicro_a9:f8:... | Broadcast      | ARP      | 60 Who has 10.10.17.1  |
| 422                                                                                    | 4.098509377  | 10.10.18.100           | 224.0.0.251    | MDNS     | 81 Standard query 0x0  |
| 423                                                                                    | 4.101222701  | 10.10.18.100           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 424                                                                                    | 4.101222883  | 10.10.18.18            | 224.0.0.251    | MDNS     | 78 Standard query 0x0  |
| 425                                                                                    | 4.101223044  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.20  |
| 426                                                                                    | 4.102233396  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.20  |
| 427                                                                                    | 4.102233752  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.1.20  |
| 428                                                                                    | 4.102233896  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.20  |
| 429                                                                                    | 4.102234036  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.23  |
| 430                                                                                    | 4.200204124  | Intel_09:6f:69         | Broadcast      | ARP      | 60 Who has 10.10.19.5  |
| 431                                                                                    | 4.200204676  | AzureWaveTec_92:a9:... | Broadcast      | ARP      | 60 Who has 10.10.18.1  |
| 432                                                                                    | 4.200204816  | AzureWaveTec_92:a9:... | Broadcast      | ARP      | 60 Who has 10.10.17.2  |
| 433                                                                                    | 4.200204958  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.2.20  |
| 434                                                                                    | 4.200205099  | DeLL_2f:0a:13          | Broadcast      | ARP      | 60 Who has 10.10.200.  |
| 435                                                                                    | 4.200205242  | CloudNetwork_b4:b2:... | Broadcast      | ARP      | 60 Who has 10.10.18.1  |
| 436                                                                                    | 4.200205384  | 10.10.19.23            | 10.10.255.255  | BROWSER  | 226 Get Backup List Re |
| 437                                                                                    | 4.200509977  | 10.10.19.23            | 10.10.19.23    | BROWSER  | 226 Get Backup List Re |
| 438                                                                                    | 4.200647284  | 10.10.141.132          | 224.0.0.251    | MDNS     | 119 Standard query res |
| 439                                                                                    | 4.200765425  | 10.10.2.35             | 10.10.19.23    | BROWSER  | 226 Get Backup List Re |
| 440                                                                                    | 4.200826887  | 10.10.13.170           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 441                                                                                    | 4.201355524  | 10.10.16.142           | 224.0.0.251    | MDNS     | 119 Standard query res |
| 442                                                                                    | 4.200826337  | 10.10.19.23            | 10.10.2.35     | NBNS     | 92 Name query NBSTAT   |
| 443                                                                                    | 4.2008948758 | 10.10.2.35             | 10.10.19.23    | NBNS     | 271 Name query respons |
| 444                                                                                    | 4.300430422  | Intel_15:2a:89         | Broadcast      | ARP      | 60 Who has 10.10.14.1  |
| 445                                                                                    | 4.302687443  | QNAP_7e:1f:2b          | Broadcast      | ARP      | 60 Who has 10.10.0.22  |

On retrouve plusieurs protocoles dont les protocoles ARP/UDP/TCP, Mais encore :

### Le Protocole mDNS :

**mDNS** est un protocole de résolution de noms sans avoir à configurer un serveur DNS. Il permet aux hôtes de communiquer entre eux dans votre réseau au sein du même VLAN sans interruption. mDNS utilise des paquets multicast pour permettre la découverte des différents hôtes dans le réseau. (couche application)

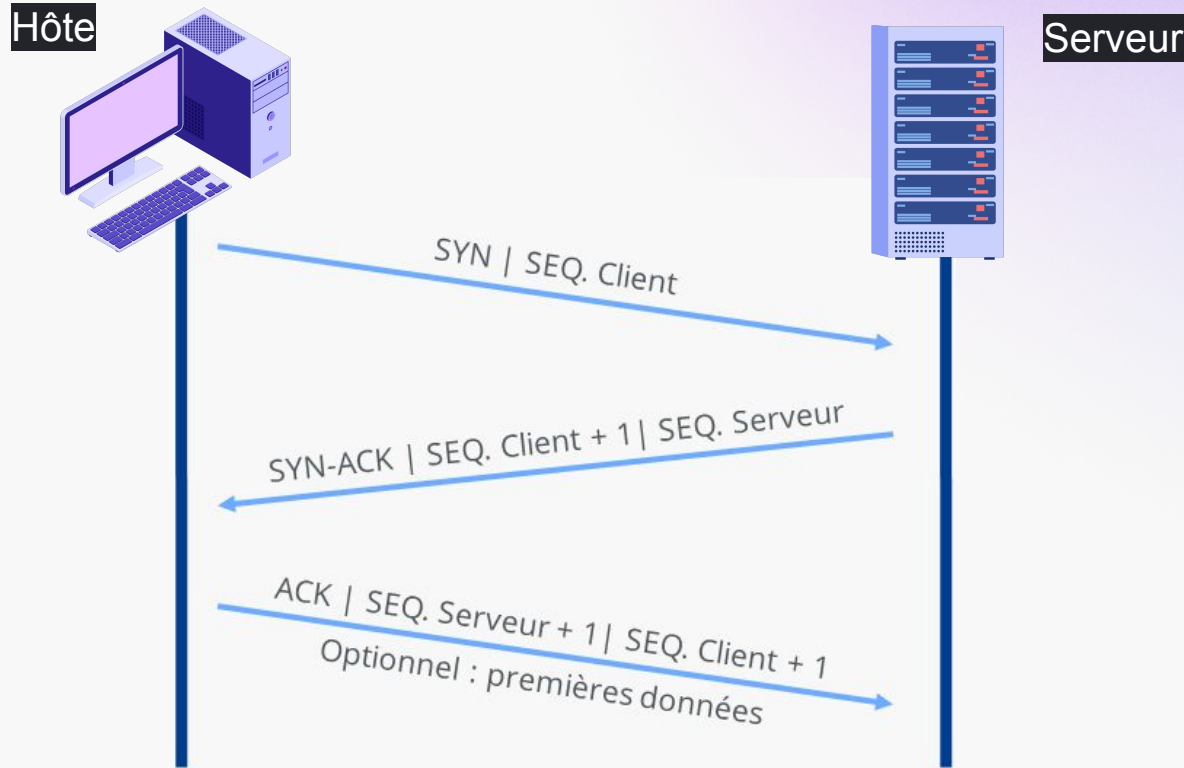
### Le Protocole QUIC :

**Quick UDP Internet Connections** » permet l'envoi rapide de paquets simples via le protocole. Équivalent de TCP + TLS en plus rapide et sécurisée, créé par Google. (Couche Transport)

### Le Protocole HTTP :

**HyperText Transfer Protocol**, est un protocole utilisé pour récupérer des ressources telles que des fichiers HTML. HTTP fournit une norme de messagerie pour faciliter les échanges de communication entre les clients Web et les serveurs Web.

## 04 Mécanisme de Connexion: three-way handshake



SEQ. = Numéro de séquence

## 04 three-way handshake

Voici comment cela fonctionne :

1. **SYN** : L'ordinateur initiateur (ou client actif) envoie un paquet de synchronisation de numéro de séquence (SYN) à l'ordinateur récepteur (généralement un serveur) avec la valeur fixée à un nombre arbitraire (par exemple J) pour « demander » si des connexions ouvertes sont disponibles.
2. **SYN-ACK** : Si l'ordinateur récepteur (ou client passif) a des ports ouverts qui peuvent accepter la connexion, il renvoie un paquet de synchronisation-accusé de réception (SYN-ACK) à l'ordinateur initiateur. Le segment SYN prend une nouvelle valeur arbitraire (par exemple K) et le segment ACK prend la valeur arbitraire d'origine plus un (J+1).
3. **ACK** : L'ordinateur initiateur (client actif) envoie ensuite un paquet de numéro de séquence d'accusé de réception (ACK) à l'ordinateur récepteur, accusant réception du paquet SYN-ACK. La valeur du paquet est fixée à la valeur arbitraire plus un (K+1). Avec cette dernière étape, la connexion est établie et la transmission des données peut commencer.

**Le three-way handshake** est un pilier du protocole **TCP**, car il établit les paramètres de connexion entre deux ordinateurs avant que la transmission des données ne commence. Une grande partie de notre activité en ligne utilise le three-way handshake.

En écoutant des échanges FTP sans TLS, que remarquez-vous dans les paquets ?  
Est-il possible de récupérer des données sensibles de connexion ?  
En est-il de même avec les échanges SSL ?

| Élément              | FTP (sans TLS)     | FTPS (avec TLS) |
|----------------------|--------------------|-----------------|
| Login / mot de passe | ✓ Visible en clair | ✗ Chiffré       |
| Commandes FTP        | ✓ Visible en clair | ✗ Chiffré       |
| Contenu des fichiers | ✓ Visible en clair | ✗ Chiffré       |
| IPs / ports          | ✓ Visible          | ✓ Visible       |
| Certificat serveur   | ✗ Inexistant       | ✓ Visible       |

## Conclusion

FTP sans TLS est **fondamentalement non sécurisé**. N'importe qui sur le même réseau peut capturer des credentials en quelques secondes. C'est pourquoi il est remplacé par **FTPS** (FTP over TLS) ou **SFTP** (FTP over SSH) dans tout environnement sérieux.

# Les différentes commandes tshark

## Lister les interfaces disponibles tshark -D

```
Fichier Éditer Affichage Recherche Terminal Aide
neyguib@localhost:~$ tshark -D
1. wlo1
2. vmnet1
3. vmnet8
4. any
5. lo (Loopback)
6. enp3s0
7. bluetooth0
8. bluetooth-monitor
9. nflog
10. nfqueue
11. dbus-system
12. dbus-session
13. ciscodump (Cisco remote capture)
14. dpauymon (DisplayPort AUX channel monitor capture)
15. randpkt (Random packet generator)
16. sdjournal (systemd Journal Export)
17. sshdump (SSH remote capture)
18. udppump (UDP Listener remote capture)
19. wifidump (Wi-Fi remote capture)
neyguib@localhost:~$
```

## Capturer sur une interface sudo tshark -i

```
Fichier Éditer Affichage Recherche Terminal Aide
Air de Chaimae_companion-link_tcp.local PTR MacBook d'Hug0_companion-link_tcp.local PTR gogeta_companion-link_tcp.local
178 1.848737182 10.10.32.142 → 224.0.0.251 MDNS 196 Standard query response 0x0000 PTR Nats's MacBook Air_companion-link_tcp.local TXT
179 1.850165525 10.10.30.183 → 224.0.0.251 MDNS 442 Standard query response 0x0000 PTR MacBook d'Hug0_companion-link_tcp.local TXT, cache flush SRV, cache flush 0 0 49435 MacBook-dh
ug0.local AAAA, cache flush fe08::cd3:769d:6c2:6908 A, cache flush 10.10.30.183 NSEC, cache flush MacBook d'Hug0_companion-link_tcp.local NSEC, cache flush MacBook-dhug0.local
180 1.945153900 Intel_a6:82:0e → Broadcast ARP 60 ARP Announcement for 10.10.32.138
181 1.945220038 c6:de:41:26:86:dd → Broadcast ARP 42 Who has 10.10.32.142? Tell 10.10.32.68
182 1.945523045 c6:de:41:26:86:dd → Broadcast ARP 42 Who has 10.10.32.210? Tell 10.10.32.68
183 1.945722675 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.3.202? Tell 10.10.0.200
184 1.945973193 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.2.203? Tell 10.10.0.200
185 1.946192064 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.224? Tell 10.10.0.200
186 1.946370558 AzureWaveTec_d8:46:37 → Broadcast ARP 60 Who has 10.10.29.254? Tell 10.10.32.94
187 1.947227686 SuperMicroCo_1f:7b:2e → Broadcast ARP 60 Who has 10.10.200.17 Tell 10.10.200.2
188 1.947617677 SuperMicroCo_1f:7b:2e → Broadcast ARP 60 Who has 10.10.200.17 Tell 10.10.200.3
189 1.947867099 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.1.217? Tell 10.10.0.200
190 1.948089196 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.227? Tell 10.10.0.200
191 1.948343844 HewlettPacka_88:5c:88 → Broadcast ARP 60 Who has 10.10.200.17 Tell 10.10.200.6
192 1.948659500 10.10.32.252 → 10.10.255.255 MDNS 92 Name query NB POSTGRES-AUTH=000
193 1.950188487 10.10.32.219 → 224.0.0.251 MDNS 206 Standard query response 0x0000 PTR andré's MacBook Air_companion-link_tcp.local TXT
194 1.950907076 10.10.31.0 → 224.0.0.251 MDNS 83 Standard query 0x0000 PTR ocularus.sp_tcp.local, "QM" question
195 1.951553188 10.10.31.0 → 224.0.0.251 MDNS 86 Standard query 0x0000 PTR ocularus.sp_v2_tcp.local, "QM" question
196 1.952874034 10.10.31.0 → 224.0.0.251 MDNS 295 Standard query response 0x0000 PTR Oroitzi:OroitziPC_ocularus.sp_tcp.local SRV 0 0 5314 OroitziPC.local TXT A 10.10.31.0 NSEC OroitziPC.local
197 1.954151614 10.10.31.0 → 224.0.0.251 MDNS 384 Standard query response 0x0000 PTR Oroitzi:OroitziPC_ocularus.sp_v2_tcp.local SRV 0 0 5312 OroitziPC.local TXT A 10.10.31.0 NSEC OroitziPC.local
198 1.955312846 10.10.32.252 → 224.0.0.251 MDNS 80 Standard query 0x0000 A postgres-appointment, "QM" question
199 1.956590793 10.10.32.252 → 224.0.0.251 MDNS 88 Standard query 0x0000 AAAA postgres-appointment, "QU" question
200 2.047562536 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.225? Tell 10.10.0.200
201 2.047739272 Intel_9d:f4:2c → Broadcast ARP 60 Who has 10.10.32.126? (ARP Probe)
202 2.048891909 LiteonTechno_c1:31:db → Broadcast ARP 60 Who has 10.10.29.227? Tell 10.10.30.50
203 2.048982099 Apple_10:05:7f → Broadcast XID 60 Basic Format: Type 1 LLC (Class 1 LLC) Window Size 0
204 2.049381342 fe08::c40f:25a5:ee90:7f56 - ff02::1 ICMV6 86 Neighbor Advertisement f080::c40f:25a5:ee90:7f56 (ovr) is at 10:51:07:9d:f4:2c
205 2.051583810 10.10.30.125 → 224.0.0.251 MDNS 437 Standard query response 0x0000 PTR MacBook Air de Abd-ellah_companion-link_tcp.local TXT, cache flush SRV, cache flush 0 0 59677 MacBook-Air-de-Abd-ellah.local AAAA, cache flush fe08::1cfb:f6e9:2d7b:ed68 A, cache flush 10.10.30.125 NSEC, cache flush MacBook Air de Abd-ellah_companion-link_tcp.local NSEC, cache flush MacBook-Air-de-Abd-ellah.local
206 2.051825860 10.10.32.204 → 10.10.255.255 UDP 86 57621 → 57621 Len=44
207 2.052113177 10.10.32.68 → 224.0.0.251 MDNS 188 Standard query 0x0000 TXT Admin's MacBook Pro_companion-link_tcp.local, "QM" question
208 2.150823813 AzureWaveTec_03:9b:90 → Broadcast ARP 60 Who has 10.10.09.79? Tell 10.10.32.67
209 2.150903668 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.230? Tell 10.10.0.200
210 2.150879069 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.1.203? Tell 10.10.0.200
211 2.151089737 Dell_2f:a0:13 → Broadcast ARP 60 Who has 10.10.200.17 Tell 10.10.200.5
212 2.151790541 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0xdffa1860
213 2.152233833 ActionsMicro_a9:f8:91 → Broadcast ARP 60 Who has 10.10.32.24? Tell 10.10.26.37
214 2.152467299 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.209? Tell 10.10.0.200
215 2.152698583 QNAP_7e:1f:2b → Broadcast ARP 60 Who has 10.10.0.205? Tell 10.10.0.200
```

## Autres commandes utile tshark

Voir uniquement les requêtes HTTP

```
sudo tshark -i [interface] -Y "http.request"
```

Voir les credentials FTP en clair

```
sudo tshark -i [interface] -Y "ftp.request.command == USER or ftp.request.command == PASS"
```

Voir le trafic DNS

```
sudo tshark -i [interface] -Y "dns"
```

Sauvegarder dans un fichier .pcap

```
sudo tshark -i ens33 -w capture.pcap
```

Lire et analyser un fichier .pcap

```
tshark -r capture.pcap
```

Lire avec un filtre

```
tshark -r capture.pcap -Y "ftp" tshark -r capture.pcap -Y "http" tshark -r capture.pcap -Y "ip.addr == [adresse ip]"
```

## Conclusion :

Wireshark et TShark sont des outils incontournables pour tout professionnel du réseau. L'un offre une interface graphique intuitive pour l'analyse interactive, l'autre une puissance en ligne de commande idéale pour l'automatisation et les environnements sans GUI. Ensemble, ils permettent de capturer, filtrer et analyser le trafic réseau en temps réel.

Comme démontré avec FTP, ils révèlent concrètement les dangers des protocoles non chiffrés et l'importance du chiffrement des communications. Ce sont des outils au service de la sécurité.

